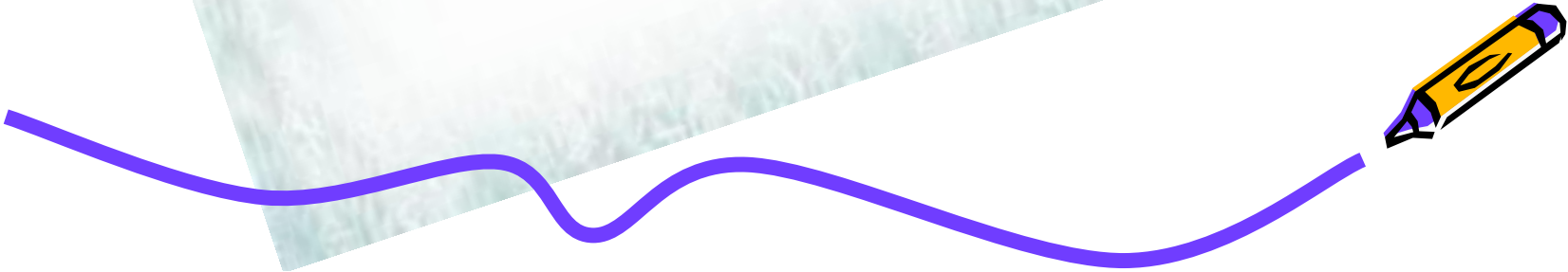




Chapter 3 Basic SQL Query Language



SQL Language-Introduction



SQL (Structured Query Language) is a language that allows us to query and manipulate data on computerized relational database systems. (

SQL is currently in transition from the relational form (the ANSI SQL-92 standard) to a newer object-relational form (the ANSI SQL-99 standard) , SQL-99 is as extending SQL-92.

SQL Language - Introduction



■ SQL语言的组成

- **Data Definition Language (DDL)**: 用于定义SQL基本表、视图、索引等。
- **Data Control Language (DCL)**: 基本表和视图的授权、完整性规则的描述和事务控制语句等。
- **Data Manipulation Language (DML)**: 分为数据查询和数据更新(插入、删除和更新)两大类操作。

SQL功能		动词
SQL DDL		CREATE、ALTER、DROP
SQL DML	数据查询	SELECT
	数据更新	INSERT、UPDATE、DELETE
SQL DCL		GRANT、REVOKE

SQL Language-Introduction



■ SQL

- 字符：单字节字符可以是字母（**A~Z**，**a~z**，**\$**，**#**，**@**或者是扩展字符集成员）、数字（**0~9**）或者是特殊字符（**，**，*****，**+**，**%**及许多其他符号）；
- 标识符：是用于构成名称的标记(一个或多个字符的序列)；
- 数据类型：决定了**DBMS**如何解释一个值；
- 常量：指定一个值，可以分为字符型、整数等数值常量。
- 运算符
- 变量与函数
- 语法规则规定

SQL Language – Introduction



■ 标识符

- 常规标识符：不需要使用双引号” ”或方括号[]等定界符进行分隔的标识符。

```
SELECT * FROM sales
```

```
CREATE DATABASE MyDatabase
```

- 常规标识符的命名规则：

- 标识符长度可以为1~128个字符
- 首字符必须为字母、下划线、@或#
- 第一个字符后面的字符可以是字母、数字、下划线、@或#等符号
- 标识符不能与SQL中的保留字同名
- 保留字与标识符等都不区分大小写

SQL Language – Introduction



■ 标识符

- **定界标识符**：在定义的标识符中用到数据库保留字、空格等特殊字符时，必须使用定界标识符，需要使用双引号” ”或方括号[]等定界符作为分隔的标识符。
- 在定界符中可以包含的特殊字符

~	-	!	%
^	'	&	.
\	`	{	}
(	)		



SQL Language – DataType



■ 数据类型—SQL Server

- 在SQL Server 2000中，数据类型是指列、存储过程参数、表达式、局部变量的数据特性，它决定了数据的存储格式，代表着不同的信息类型。
- 数据类型分为：系统数据类型和用户定义数据类型两种。
- 用户定义数据类型可以使用系统存储过程sp_addtype建立，也可以在企业管理器窗口采用交互式方式建立。

SQL Language – DataType



■ 系统数据类型

■ SQL Server所提供的系统数据类型如下表所示：

数据类型名称	定义标识
日期时间型	datetime, smalldatetime
整数型	int, smallint, tinyint
精确数值型	decimal, numeric
近似数值型	float, real
货币型	money, smallmoney
位型	bit
字符型	char, varchar, text
Unicode 字符串	nchar, nvarchar, ntext
二进制数据类型	binary, varbinary, image
特殊数据类型	cursor, uniqueidentifier, timestamp

SQL Language – DataType



■ 系统数据类型—日期时间型

- 日期时间型数据代表日期和时间，这种数据类型分为 **datetime** 和 **smalldatetime** 两种，二者的区别主要在于它们的存储长度、表示的时间范围和精度不同。在输入日期时间型数据时应将数据放在单引号中。

	datetime	smalldatetime
存储长度	8字节	4字节
精度	3/100秒	1分钟
最小值	1753年1月1日	1900年1月1日
最大值	9999年12月31日	2079年6月6日

SQL Language – DataType



■ 系统数据类型—日期时间型

- **datetime**类型：4字节用来存储日期，表示距1900年1月1日之前或之后的天数，值为负，表示之前。另外4字节用来存储距12:00AM（午夜）的毫秒数。
- **smalldatetime**类型：2字节表示距1900年1月1日的天数，另外2字节表示距12:00AM（午夜）的分钟数。
- 与**datetime**数据类型相比，**smalldatetime**类型占用的存储空间小，但它表示的日期范围也小，时间精度较低。
- SQL Server中没有单独的时间数据类型或日期数据类型，若输入数据不带时间，则时间默认为数据日期当天的午夜；若输入数据不带日期，则日期默认为**1900年1月1日**。

SQL Language – DataType



■ 系统数据类型—日期时间型

- SQL Server可识别的日期格式有三种：**字符格式**、数字格式和无分隔符字符串格式。

字符格式

October 2, 2000

October 2, 00

Oct 2, 2000

Oct 2, 00

2000, October 2

00, Oct 2

在字符格式中，如果只用2位数表示年份，则在其值小于50时，系统将它翻译为**20****年；如果年份大于或等于50，则将它解释为**19****年。

SQL Language – DataType



■ 系统数据类型—日期时间型

- SQL Server可识别的日期格式有三种：字符格式、**数字格式**和无分隔符字符串格式。

数字格式

6/15/2000 (月/日/年)

06/15/2000 (月/日/年)

06/15/00 (月/日/年)

06-15-00 (月/日/年)

15-06-2000 (日/月/年)

数字格式的日期串中年、月、日均用数字表示，相互间用“/”、“-”或“.”分隔，数字格式的默认顺序为**mdy**。用户可以用**set dateformat**语句重设格式。

SQL Language – DataType



■ 系统数据类型—日期时间型

- SQL Server可识别的日期格式有三种：字符格式、数字格式和**无分隔符字符串格式**。
 - 无分隔字符串格式，用**4位**、**6位**或**8位**字符串表示，数据之间不能有分隔符。对于**6位**和**8位**字符串，它所表达的日期格式为年、月、日，其中月和日必须各占两位，如：“20001231”或“001231”。
 - 用**4位**表示时，它只代表年份，其月和日默认为**1月1日**，例如：“2004”，表示“2004年1月1日”。

SQL Language – DataType



■ 系统数据类型—整数型

- SQL Server中有四种整数类型：**bigint**、**int**、**smallint**和**tinyint**

整数数据类型	存储长度	最小值	最大值
tinyint	1字节	0	255
smallint	2字节	-32768	32767
int	4字节	-2147483648	2147483647
bigint	8字节	-92233720368 54775808	92233720368 54775807

SQL Language – DataType



■ 系统数据类型—精确数值型

- 精确数值型数据由整数和小数两部分组成。SQL Server提供了两种精确数值类型：**decimal[(p[, s])]** 和 **numeric[(p[, s])]**
- 精确数值型能表达的数据范围是： $-10^{38} \sim 10^{38}-1$

精度(位)	字节长度
1~9	5
10~19	9
20~28	13
29~38	17

在声明精确数值型时，可以定义数据的精度**p(precision)**和小数位数**s(scale)**。精度表示所存十进制数据的位数总和，包括整数和小数部分。

SQL Language – DataType



■ 系统数据类型—浮点数

- **REAL** 4 个字节，单精度浮点数，负数范围是从 $-3.402823E38$ 到 $-1.401298E-45$ ，正数从 $1.401298E-45$ 到 $3.402823E38$ ，和 0。
- **FLOAT** 8 个字节，双精度浮点数，负数范围是从 $-1.79769313486232E308$ 到 $-4.94065645841247E-324$ ，正数从 $4.94065645841247E-324$ 到 $1.79769313486232E308$ ，和 0。

SQL Language – DataType



■ 系统数据类型—货币型

- 货币数据类型有**money**和**smallmoney**两种。
- **money**型数据为8字节长，而**smallmoney**型数据的长度是4字节。
- 货币型数据以十进制数表示货币量。

■ 系统数据类型—位数据类型

- 位数据类型用**bit**关键字声明，数据有**0**、**1**两种取值。
- 输入**0**以外的其它值时，系统均把它们当作**1**看待。
- 这种数据类型常作为逻辑变量使用，用于表示真、假，或是、否等二值选择。

SQL Language – DataType



■ 系统数据类型—字符类型

- 字符数据由字母、符号和数字组成，在输入字符数据时应将数据放在单引号 ‘ ’ 内
- 字符型数据也有定长和变长两种，分别是char和varchar。
- Char[(n)]类型长度是固定的，当数据长度小于给定n值时，输入字符串的后面将被填充空格。它的取值范围是1~8000。
- Varchar[(n)]类型的长度是可变的，n最大值为8000，如果不指定n值，则系统默认长度为1。
- 文本Text类型也是一种可变长度数据类型，定义这类数据类型不需要指定长度。数据最大可达 $2^{31}-1$ ，用于存储长度大于8KB的字符数据。
- 向text列插入数据时，应将数据放在单引号内。

SQL Language – DataType



■ 系统数据类型—Unicode类型

- 该类型用于支持国际上的非英语语种，每个Unicode字符的存储长度为2字节
- Unicode字符类型也有：**nchar**、**nvarchar**和**ntext**三种。其中**nchar[(n)]**，**nvarchar[(n)]**，n最大长度为4000个字符。**ntext**常用于存储长度大于4000的变长Unicode字符
- 对于Unicode字符常量数据，应在前面加上标识符**N**，但是存储时并不存储该字符。

SQL Language – DataType



■ 系统数据类型—二进制数据类型

- SQL Server支持的二进制数据类型包括**binary**、**varbinary**和**image**三种。二进制数据常用于存储图象、有格式的文本(如: Word文件、Excel文件等)、程序文件数据等。
- **binary[(n)]**数据的存储长度是固定的, 即**n+4**。当输入的二进制数据长度小于**n**时, 余下部分填充0
- **varbinary[(n)]**数据的存储长度是可变的, 它是实际所输入数据的长度加上4字节
- 以上两种数据类型的最大长度都是8000
- **image**也是一种变长二进制数据类型, 其最大长度可为 $2^{31}-1$

SQL Language – DataType



■ 系统数据类型—时间戳数据类型

- 时间戳数据类型是一种自动记录时间的数据类型，主要用于在数据表中记录其数据的修改时间。用关键字 **timestamp** 表示，**timestamp** 数据类型与时间和日期无关，是二进制数字，它表明数据库中数据修改发生的相对顺序。
- 一个数据库的当前时间戳可用 **@@DBTS** 函数读取。
- 当数据表中定义有时间戳列时，如果对其数据使用 **insert**、**Update**、**Delete** 操作，则系统自动将数据库的当前时间戳插入到时间戳列中
- 每个表中只能有一个时间戳列、在表中不要将时间戳列作为主键或外键

SQL Language – DataType



■ 用户定义数据类型(UDDT, User-defined datatype)

- 可以根据系统数据类型来生成用户定义的数据类型
- 分别使用`sp_addtype`生成/`sp_droptype`删除用户定义数据类型，如果系统数据类型表达式需要用到圆括号，则要放在引号内。如果任何表使用用户定义数据类型，则不能删除这个用户定义数据类型。

例如：Exec sp_addtype phone, 'char(13)'

Exec sp_addtype ssn, 'VARCHAR(11)', 'NOT NULL'

Exec sp_addtype birthday, datetime, 'NULL'

- 也可以通过企业管理器(Enterprise Manager)增加用户定义数据类型



SQL Language – Operators



■ 运算符

- 运算符用来执行列、常量或变量之间的数学运算和比较操作
- 在SQL语言中的运算符分为算术运算符、位运算符、比较运算符、逻辑运算符、联接运算符和赋值运算符等。

■ 运算符-算术运算符

- 算术运算符用于执行数字型表达式的算术运算
 - +：加或正号
 - -：减或负号
 - *：乘
 - /：除
 - %：取模，返回两个整数相除的余数

SQL Language-Operators



■ 运算符—算术运算符

■ 各种算术运算符所操作的数据类型

运算符	数据类型
+, -, *, /	int, smallint, tinyint, numeric, decimal, float, real, money, smallmoney
%	int, smallint, tinyint

SQL Language-Operators



■ 运算符一位运算符

- 位运算符对整数或二进制数据进行按位与(&)、或(|)、异或(^)、求反(~)等逻辑运算

左操作数	右操作数
binary, varbinary	int, smallint, tinyint
int, smallint, tinyint	int, smallint, tinyint, binary, varbinary
bit	int, smallint, tinyint, bit

- 求反运算(~)是一个单目运算，它只能对int, smallint, tinyint或bit类型的数据进行求反运算

SQL Language – Operators



■ 运算符—比较运算符

- 在Transact-SQL语言中，比较运算符能够进行除text、ntext和image数据类型之外的其它数据类型表达式的比较操作，返回值为布尔数据类型，即true、false

- >大于 =等于 <小于
- >=大于等于 <=小于等于 <>、!=不等于
- !>不大于 !<不小于
- [NOT] LIKE :执行样式匹配（通常只限于字符数据类型）。
- IS [NOT] NULL :测试列的内容或表达式的结果是否为空。
- BETWEEN *expr1* AND *expr2* :值的测试范围。
- *expr1* [NOT] IN (*val1*, *val2*, ...)或(*subquery*) : 测试 *expr1* 在值是否在列表中或是在子查询 的结果集中。
- ANY (SOME) : 测试子查询的结果集中是否有一行或多行满足指定的条件。（ANY 和 SOME 是同义词）
- ALL: 测试子查询结果集的所有行是否都满足指定的条件。
- [NOT] EXISTS: 测试子查询是否返回任何结果。

SQL Language – Operators



■ 运算符—逻辑运算符

■ 逻辑运算符用于测试条件是否为真，根据测试结果返回布尔值 **TURE**、**FALSE**或**UNKNOWN**

- **AND**

- 对两个布尔表达式的值进行逻辑与运算，如果其中有一个为**TURE**，另一个为**UNKNOWN**，或两个都为**UNKNOWN**，则返回值为**UNKNOWN**

- **OR**

- 对两个布尔表达式的值进行逻辑或运算，如果其中有一个为**FALSE**，另一个为**UNKNOWN**，或两个都为**UNKNOWN**，返回**UNKNOWN**

- **NOT**

- 对布尔表达式的值进行取反运算，当其值为**UNKNOWN**时，返回值为**UNKNOWN**

SQL Language – Operators



■ 运算符—运算符的优先级

- ✓ + (正)、- (负)、~ (按位 NOT)
- ✓ * (乘)、/ (除)、% (模)
- ✓ + (加)、- (减)
- ✓ =, >, <, >=, <=, <>, !=, !>, !< 比较运算符
- ✓ ^ (位异或)、& (位与)、| (位或)
- ✓ NOT
- ✓ AND
- ✓ ALL、ANY、BETWEEN、IN、LIKE、OR、SOME
- ✓ = (赋值)



SQL Language – 变量和函数



■ 变量

- 在SQL语言中，变量分为局部变量和全局变量。局部变量由用户定义和维护，而全局变量则由系统定义和维护
- 局部变量以单个@字符开头，全局变量以@@开头

■ 变量—局部变量

- 局部变量用DECLARE声明，它只能用在声明该变量的批或过程体内。
- 格式：
 - DECLARE

```
{ { @local_variable data_type}
  | {@cursor_variable_name CURSOR}
  | {table_type_definition}
}
```

SQL Language – 变量和函数



■ 参数

- **@local_variable**: 是变量的名称。变量名必须以@开头。局部变量名必须符合标识符规则。
 - **data_type**: 是任何由系统提供的或用户定义的数据类型。变量不能是 **text**、**ntext** 或 **image** 数据类型。
- **@cursor_variable_name**: 是游标变量的名称。游标变量名必须以@开头并遵从标识符规则。
 - **CURSOR**: 指定变量是局部游标变量。
- **table_type_definition**: 定义表数据类型。表声明包括列定义、名称、数据类型和约束。允许的约束类型只包括 **PRIMARY KEY**、**UNIQUE KEY**、**NULL** 和 **CHECK**。

SQL Language – 变量和函数



■ 变量和函数—局部变量

■ 声明局部变量后，可用**SELECT**或**SET**语句为其赋值

■ 格式：

- **SELECT**

SELECT @local_variable=expression

- **SET**

SET @local_variable=expression

■ 例如：

Declare @var1 char(4), @var2 char(4), @var3 char(8)

Set @var1='abcd'

Set @var2='efgh'

Select @var3=@var1+@var2

select @var1,@var2,@var3 (输出)

SQL Language – DDL



- AS子句可用来更改结果集列名或为导出列指定名称。

```
Declare @var1 char(4), @var2 char(4), @var3 char(8)
```

```
Set @var1='abcd'
```

```
Set @var2='efgh'
```

```
Select @var3=@var1+@var2
```

```
select @var1 as "var1",var2=@var2,@var3 as var3
```


SQL Language – 变量和函数



■ 函数

- 函数用于在数据上执行某些操作，如修改显示的数据、转换数据类型或进行计算等
- 返回表中查询的每一行的值，既可用在**SELECT**语句里，也可通过**WHERE**子句指出标量函数的条件表达式

■ 函数的分类

- 聚合函数
- 标量函数
- 行集函数

聚合函数



- 聚合函数（多行函数）将查询结果中多行值进行汇总。这些函数执行时，对表中一列或多列的值进行汇总并产生单一的值。

- 语法

```
SELECT FUNCTION_NAME(COLUMN_NAME)
FROM TABLE_NAME
WHERE CONDITION
```

- 举例：

- SUM, AVG, COUNT, MAX, MIN

- 思考：查询考试最高分和最低分，查询学生的总人数

标量函数



■ 标量函数（单行函数）

■ 语法： **SELECT function_name[(arg1,arg2,...)]**

■ 分类

- 字符串函数
- 数学函数
- 日期和时间函数
- 元数据函数
- 安全函数
- 系统函数
- 系统统计函数

标量函数



■ 字符串函数

- 获得字符的**ASCII**码，返回指定个数的字符，转换字符大小写
- 举例
 - **SELECT ASCII('A')**返回字符的**ASCII**码
 - **SELECT CHAR(97)****ASCII**的逆函数
 - **SELECT NCHAR(23425)**返回给定整数对应的**Unicode**字符
 - **SELECT UNICODE('宁')**返回字符**UNICODE**编码值
 - **SELECT LEFT('BigCollege',3)**返回从字符串左边开始指定个数的字符串，其逆函数是**RIGHT**
 - **SELECT UPPER('BigCollege')**转换为大写字符，逆函数是**LOWER**
 - **SELECT LEN('BIGCOLLEGE')**返回字符个数，不包含尾随空格

标量函数



■ 字符串函数

■ 举例

- `SELECT REPLACE('BIGCOLLEGE','BIG','SMALL')` 替换
- `SELECT STR(2.628) AS S1, STR(2.628,4,2) AS S2 , STR(2.628,5,3) AS S2` 将数值转换为字符
- `SELECT SUBSTRING('BIGCOLLEGE',4,4)` 截取指定位置开始的指定长度的字符串
- . . .

■ 思考

- 在 **Department** 表中查询所有系的信息，并且系主任的名字只显示其姓。

标量函数



■ 数学函数

■ 对数值型数据进行数学运算

■ 举例

- **SELECT COS(14.78)**
- **SELECT CEILING(-3.66)**返回大于或等于指定数值表达式的最小整数，逆函数为**FLOOR**
- **SELECT LOG10(100)**
- **SELECT PI()**返回圆周率
- **SELECT RAND()**返回0到1之间的随机float值
- **SELECT ROUND(33.66,1)**四舍五入到指定的长度

■ 思考：显示成绩表中学生的成绩，四舍五入到十位。

标量函数



■ 日期和时间函数

■ 用于显示关于日期和时间的信息

■ 举例：

- **SELECT GETDATE()**返回当前日期
- **Select CONVERT(varchar(30),getdate(),101)**将某种数据类型显式转换为另一种数据类型，**CAST** 提供相似的功能。
- **SELECT DATENAME(YEAR,'10/12/2010'), DATENAME(MONTH,'10/12/2010')**返回指定日期部分的字符串，**DATEPART**函数返回数值
- **SELECT YEAR('2/10/2010')**返回年份的整数
- **SELECT DATEdiff(month,'2/26/2010','7/13/2010')**

■ 思考：找出上课时间少于4个月的课程

行集函数



- 行集函数返回对象，该对象可在SQL语句中用作表引用。
- 行集函数主要有6个：
 - FREETEXTTABLE (全文索引)
 - CONTAINSTABLE (全文索引)
 - OPENXML (XML文档)
 - OPENDATASOURCE
 - OPENQUERY
 - OPENROWSET

SQL Language – Introduction



■ SQL语法格式约定

- 大写字母：代表保留的关键字
- 小写字母：表示对象标识符、表达式等
- {}: 大括号中的内容必须为必选参数，其中的多个选项用|隔开。
- []: 方括号表示列出的选项是可选的，用户可以不选择
- |: 竖线表示参数之间的“或”关系
- 省略号.....: 表示重复前面语法单元
- 注释
 - 单行注释: “--”表示
 - 块注释: /*.....*/表示

3.2 Setting Up the Database



■ 用SQL生成数据库

- **ON:** 显式定义用来存储数据库数据部分的磁盘文件（数据文件）

<filespec>: 定义数据文件

<filegroup>: 定义数据文件组

- **LOG ON:** 显式定义用来存储数据库日志的磁盘文件（日志文件*.ldf），如果没有指定该选项，则系统自动创建一个日志文件

- **COLLATE:** 指定数据库的默认排序规则

- **FOR LOAD:** 与早期版本的SQL Server 兼容

- **FOR ATTACH:** 指定从现有的一组操作系统文件中附加数据库

```
CREATE DATABASE database_name
[ ON
    [ < filespec > [ ,...n ] ]
    [ , < filegroup > [ ,...n ] ]
]
[ LOG ON { < filespec > [ ,...n ] } ]
[ COLLATE collation_name ]
[ FOR LOAD | FOR ATTACH ]
```

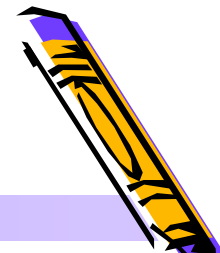
其中: **< filespec > ::=**

```
[ PRIMARY ]
( [ NAME = logical_file_name , ]
  FILENAME = 'os_file_name'
  [ , SIZE = size ]
  [ , MAXSIZE = { max_size |
    UNLIMITED } ]
  [ , FILEGROWTH =
    growth_increment ] ) [ ,...n ]
```

< filegroup > ::=

```
FILEGROUP filegroup_name < filespec >
[ ,...n ]
```

3.2 Setting Up the Database



■ 用SQL生成数据库

- **PRIMARY**: 主文件组。指定关联的 **<filespec>** 列表定义主文件。一个数据库只能有一个主文件。如果没有指定 **PRIMARY**, 那么 **CREATE DATABASE** 语句中列出的第一个文件将成为主文件 (*.mdf)。主文件包含在主文件组中。
- **FILEGROWTH**: 指定 **<filespec>** 中定义的文件的生长增量。文件的 **FILEGROWTH** 设置不能超过 **MAXSIZE** 设置。
- **FILEGROUP**: 用户定义的文件组。

```
CREATE DATABASE database_name  
[ ON  
    [ < filespec > [ ,...n ] ]  
    [ , < filegroup > [ ,...n ] ]  
]  
[ LOG ON { < filespec > [ ,...n ] } ]  
[ COLLATE collation_name ]  
[ FOR LOAD | FOR ATTACH ]
```

其中: < filespec > ::=

```
[ PRIMARY ]  
( [ NAME = logical_file_name ,  
    FILENAME = 'os_file_name'  
    [ , SIZE = size ]  
    [ , MAXSIZE = { max_size |  
UNLIMITED } ]  
    [ , FILEGROWTH =  
growth_increment ] ) [ ,...n ]
```

< filegroup > ::=

```
FILEGROUP filegroup_name  
< filespec > [ ,...n ]
```

3.2 Setting Up the Database



Create DataBase Student_DB ON Primary

(Name=Student_DB_dat,
FileName='d:\data\Student_DB.mdf'

Size=10MB,

MaxSize=50MB,

FileGrowth=15%),

FileGroup Student_DB_Data

(Name=Student_DB_Data_dat,
FileName='e:\data\Student_DB_Data.ndf',

Size=50MB,

MaxSize=100MB,

FileGrowth=10MB)

CREATE DATABASE *database_name*
[**ON**

[< filespec > [,...*n*]]

[, < filegroup > [,...*n*]]

]

[LOG ON { < filespec > [,...*n*] }]

[COLLATE *collation_name*]

[FOR LOAD | FOR ATTACH]

< filespec > ::=

[**PRIMARY**]

([NAME = *logical_file_name* ,]

FILENAME = '*os_file_name*'

[, SIZE = *size*]

[, MAXSIZE = { *max_size* |

UNLIMITED }]

[, FILEGROWTH =

growth_increment]) [,...*n*]

< filegroup > ::=

FILEGROUP *filegroup_name* < filespec >
[,...*n*]

3.2 Setting Up the Database



Log ON

```
( Name=Student_DB_log,  
  FileName='e:\log\Student_DB_log.ldf',  
  Size=50MB,  
  MaxSize=100MB,  
  FileGrowth=10MB)  
GO
```

用**Name**指定文件的逻辑名，
FileName指定实际文件名

3.2 Setting Up the Database



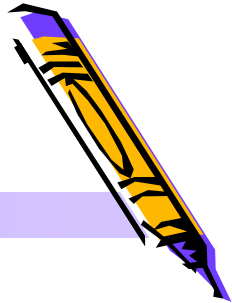
- 练习：创建名为 **Products** 的数据库，并指定单个文件为主文件，文件初始大小为**4MB**，最大可到**10MB**，文件增长大小为**1MB**。

```
CREATE DATABASE Products
ON ( NAME = prods_dat,
      FILENAME = "c:\program files\microsoft sql
server\mssql\data\prods.mdf" ',
      SIZE = 4,
      MAXSIZE = 10,
      FILEGROWTH = 1 )
```

■ CREATE DATABASE mytest

- 主数据库文件的大小为 **model** 数据库主文件的大小，事务日志文件的大小为 **model** 数据库事务日志文件的大小。因为没有指定 **MAXSIZE**，文件可以增长到填满所有可用的磁盘空间为止。

3.2 Setting Up the Database



■ 用T-SQL生成数据库

■ 文件类型

数据类型	文件扩展名
主要数据文件	.mdf
次要数据文件	.ndf
事务日志文件	.ldf

3.2 Setting Up the Database



■ 用T-SQL管理数据库

- 可以在数据库中添加或删除文件组，也可以更改文件和文件组的属性。

ALTER DATABASE *database_name*

```
{ ADD FILE <filespec> [,...n] [to FILEGROUP filegroup_name]
| ADD LOG FILE <filespec> [,...n]
| REMOVE FILE logical_file_name
| ADD FILEGROUP filegroup_name
| REMOVE FILEGROUP filegroup_name
| MODIFY FILE <filespec>
| MODIFY NAME=new_dbname
| MODIFY FILEGROUP filegroup_name {filegroup_property |
NAME=new_filegroup_name}
| SET <optionspec> [,...n][WITH <termination>]
| COLLATE <collation_name>
}
```


3.2 Setting Up the Database



- 例如：将现有文件增加到20MB，然后增加新文件

Alter DataBase Student_DB

Modify File

(Name=Student_DB_dat,
Size=20MB)

Go

Alter DataBase Student_DB

Add File

(Name=Student_DB_Data_dat2,
FileName='e:\data\Student_DB_Data2.ndf',
Size=50MB,
MaxSize=100MB,
FileGrowth=5MB)

3.2 Setting Up the Database



■ 用T-SQL管理数据库

■ 删除数据库

DROP DATABASE *database_name*

■ 将现有数据库的大小缩小到20MB

DBCC ShrinkDatabase (Student_DB,20)

- 生成与管理数据库时所做的决策对数据库的性能与可靠性影响重大，所以不是简单的使用企业管理器就能解决问题。对数据库的进一步管理将涉及到“数据库备份与恢复”、“事务管理与事务日志”、“管理超大SQL Server数据库”、“SQL Server内幕”、“数据库设计与性能”等。这些知识将在后面的章节中具体介绍

SQL Language – DDL



- SQL的数据定义功能主要包括三个部分：定义基本表、定义视图和定义索引。如下表所示：

操作对象	创 建	删 除	修 改
表	CREATE TABLE	DROP TABLE	ALTER TABLE
视图	CREATE VIEW	DROP VIEW	
索引	CREATE INDEX	DROP INDEX	

SQL Language – DDL



■ 定义、删除与修改基本表

■ 定义基本表

- **CREATE TABLE** <表名>

(<列名> <数据类型>[<列完整性约束条件>]

[,<列名> <数据类型>[<列完整性约束条件>]...

[,<表级完整性约束条件>]);

■ 约束和完整性

- **UNIQUE**: 唯一约束
- **DEFAULT**: 默认值约束
- **Primary Key**: 主键约束
- **Foreign key**: 外键约束
- **CHECK**: 检查约束:从逻辑表达式判断

SQL Language – DDL



- 学生 S(S#, SNAME, AGE, SEX)
- 课程 C(C#, CNAME, CTYPE)
- 成绩 SC(S#, C#, GRADE)

S#	SNAME	AGE	SEX
----	-------	-----	-----

C#	CNAME	CTYPE
----	-------	-------

S#	C#	GRADE
----	----	-------

SQL Language – DDL



■ 定义、删除与修改基本表

■ 学生 S(S#, SNAME, AGE, SEX)

Create Table S

(s# CHAR(10) NOT NULL UNIQUE, /*s#取值唯一*/

sname VARCHAR(20),

age TINYINT,

sex CHAR(2),

PRIMARY KEY(s#)

/*s#是主键，不能为空*/

);

SQL Language – DDL



■ 定义、删除与修改基本表

■ 课程 C(C#, CNAME, TEACH)

Create Table C

```
( c#  CHAR(10) PRIMARY KEY,  
  cname VARCHAR(20) UNIQUE,  
  teach CHAR(20),  
);
```

SQL Language – DDL



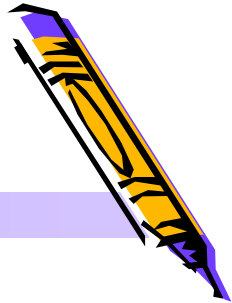
■ 定义、删除与修改基本表

■ 成绩 SC(S#, C#, GRADE)

Create Table SC

```
( s#    CHAR(10),  
  c#    CHAR(10),  
  grade decimal(5,2),  
  Primary Key(S#, C#),  
  Foreign Key(S#) References S(S#),  
  Foreign Key(C#) References C(C#),  
  Check ((grade is null) or (grade Between 0 and 100))  
)  
/*检查子句Check: 标识成绩grade或者为空, 或者是  
  0~100之间的值*/
```


SQL Language – DDL



■ 定义、删除与修改基本表

■ 练习：使用SQL语言生成基本表

Emp_no	Lname	Fname	Phone	Dept	Salary
1	Smith	John	555-1111	20	\$10 000
2	Jones	Jill	555-1211	20	\$10 000
3	Johnston	Bob	555-3214	30	\$20 000
4	Jensen	Carl	555-4321	40	\$12 000
5	Wright	Alex	555-2156	40	\$14 000
6	Ivigns	Kris	555-3215	20	\$21 000

SQL Language – DDL



Create Table Employee

```
( Emp_no int Identity(1,1) NOT NULL,  
  Lname varchar(20) NOT NULL,  
  Fname varchar(20) NOT NULL,  
  Phone char(13),  
  Dept int,  
  Salary money,  
  Primary Key(Emp_no)  
)
```

IDENTITY[(seed , increment)]:

用于获得自动增加的标识号

- **Seed:**装载到表中的第一个行所使用的值。
- **Increment:**增量值，该值被添加到前一个已装载的行的标识值上。
- 必须同时指定种子和增量，或者二者都不指定。如果二者都未指定，则取默认值 **(1,1)**。

SQL Language – DDL



■ 定义、删除与修改基本表

■ 修改基本表

ALTER TABLE <表名>

[**ADD** <新列名> <数据类型>[<列完整性约束条件>]]

[**DROP column** <列名> | <完整性约束名>]

[**ALTER** <列名> <数据类型>];

■ 修改基本表- 增加属性 “**ALTER ... ADD...**”

- 例如：在学生基本表**S**中增加一个地址属性**Address**和**City**

Alter Table **S** Add **Address** varchar(30), **City** varchar(10)

- 新增加的属性不能定义为“**NOT NULL**”，这样原来的元组在新增加的属性上的值均被置为**NULL**

SQL Language – DDL



■ 定义、删除与修改基本表

■ 修改基本表- 删除属性 “**ALTER ... DROP { [CONSTRAINT] constraint_name | COLUMN column_name }**”

- 例如：在学生基本表**S**中删除属性**age**

Alter Table **S** Drop Column **age**

- 一张表中的属性有可能被其他表引用，因此，在删除属性时通常要指明是**CASCADE**方式，还是**RESTRICT**方式，**CASCADE**表示删除属性**age**时，所有引用到属性**age**的视图、约束也要一起自动删除；**RESTRICT**表示当没有视图或约束引用**age**时，才能将其删除，否则拒绝删除操作

Alter Table **S** Drop Column **age** CASCADE

- 例如：删除基本表**S**中主键属性

SQL Language – DDL



■ 定义、删除与修改基本表

■ 修改基本表- 修改属性 “ALTER ... ALTER Column...”

- 例如：修改学生基本表**S**中的属性**sname**的字符串长度

Alter Table **S**

Alter Column **sname** varchar(25)

- 修改属性数据类型后，新数据类型必须与原数据类型相匹配，并且一般新数据类型的长度要能够存储表中已有数据

SQL Language – DDL



■ 定义、删除与修改基本表

■ 删除基本表

DROP TABLE <表名>

■ 例如：删除学生基本情况表S

Drop Table S

- 注意：基本表一旦删除，表中的数据，此表上建立的索引和视图都将自动被删除掉。

SQL Language – DDL



■ 临时表

- 在**SQL Server**中，数据表分为永久数据表和临时数据表两种，永久数据表在创建后一直存储在数据库文件中，直到用户删除为止。而临时表则在用户退出系统或修复系统时自动删除
- 临时表以**#**号开头，**一个#号**表示局部临时表，**两个#号**则表示全局临时表
- 临时表在数据库**tempdb**中生成

■ 小结：

- 表是关系数据库系统的关键，生成表时一定要认真分析，选择适当的数据类型，保证有效的数据存储，在表中增加适当限制以维护数据的完整性。并将表格生成和修改保存为脚本，以便在必要时重新生成。

Exercises in dormitory



- 1、用SQL语言创建CAP数据库和其中的四张数据表。
- 2、如何创建主外键（图形化界面）。
- 3、如何修改列名？

SQL Language - Introduction



■ SQL语言的组成

- **Data Definition Language (DDL):** 用于定义SQL基本表、视图、索引等。
- **Data Manipulation Language (DML):** 分为数据查询和数据更新(插入、删除和更新)两大类操作。
- **Data Control Language (DCL):** 基本表和视图的授权、完整性规则的描述和事务控制语句等。

SQL功能		动词
SQL DDL		CREATE、DROP、ALTER
SQL DML	数据查询	SELECT
	数据更新	INSERT、UPDATE、DELETE
SQL DCL		GRANT、REVOKE

3.10 Insert, Update, and Delete



- The three SQL statements (*update statements*) — **Insert**, **Update**, and **Delete** — are used to perform data modifications to existing tables.

- **The Insert Statement**

```
INSERT INTO tablename [ ( colname { ,colname...} ) ]  
    {VALUES (expr | null { , expr | null...}) | subquery }
```

1. 插入单个元组

- **INTO**子句中没有出现的属性列，新记录在这些列上将取空值**null**。

3.10 Insert, Update, and Delete



■ **Example 1:** 将一个新学生记录（学号：95020；姓名：陈冬；性别：男；年龄：18岁）插入到S表中。

```
INSERT INTO S
```

```
VALUES ( '95020', '陈冬', '男', 18 );
```

■ **Example 2:** 将《数据库原理与实用技术》插入到C表中，id号为c1。

■ **Example 3:** 陈冬《数据库原理与实用技术》考试成绩为90。

3.10 Insert, Update, and Delete



■ **EXAMPLE 3.10.1** Add a row with specified values(1107,aug,c006,a04,p01) to the **orders** table, setting the qty and dollars columns null.

■ **Insert into** orders(ordno,month,cid,aid,pid)
values(1107,'aug',c006,'a04','p01');

■ **Insert into** orders
values(1107,'aug',c006,'a04','p01',null,null);

3.10 Insert, Update, and Delete



■ The Insert Statement

2.插入子查询结果(在介绍了select后再讲)

■ **Example4:**在选课表SC中把平均成绩大于80分的男学生的学号和平均成绩存入另一个关系S_GRADE(S#, AVG_GRADE)中

INSERT INTO S_GRADE(S#, AVG_GRADE)

SELECT S#, AVG(GRADE)

FROM SC

WHERE S# IN

(Select S#

From S

Where Sex='男')

Group by S#

Having AVG(GRADE)>80

	s#	avg_grade
1	1	94
2	3	88
3	6	97

3.10 Insert, Update, and Delete



■ The Update Statement

UPDATE tablename

SET colname={expr | null | (subquery)}

{,colname={expr | null | (subquery)}...}

[**WHERE** search_condition];

- 修改指定表中满足**WHERE**子句条件的元组。其中**SET**子句给出 **expr | null | (subquery)** 的值用于取代相应的属性列值。如果省略 **WHERE** 子句，则表示要修改表中的所有元组。

1. 修改某一个元组的值

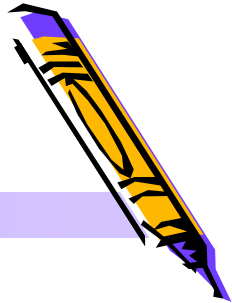
- **Example1:** 将学号为95020的学生的C1课程的成绩改为98。

Update SC

Set grade=98

Where S#='95020' and C#='C1'

3.10 Insert, Update, and Delete



■ The Update Statement

- **EXAMPLE3.10.3:** Give all agents in New York a 10% raise in the percent commission they earn on an order.

update agents

set percent=1.1*percent

where city ='New York'

2.修改多个元组的值

- **Example2:** 将所有学生的所有课程成绩提高10%

Update SC

Set grade=grade*1.1

3.10 Insert, Update, and Delete



■ The Update Statement

3.修改带子查询的语句(在介绍了select后再讲)

- **Example3:** 在基本表SC中，把课程名为数据库原理与实用技术的成绩提高2%

Update SC

SET grade=grade*1.02

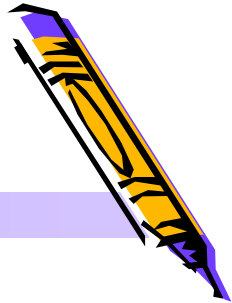
Where C# IN

(**SELECT** C#

FROM C

Where Cname='数据库原理与实用技术')

3.10 Insert, Update, and Delete



■ The Update Statement

3. 修改带子查询的语句(在介绍了select后再讲)

- **Example4:** 当学生的C3课的成绩低于该门课程的平均成绩时, 将其成绩提高5%

Update SC

SET grade=grade*1.05

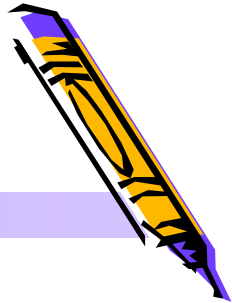
Where C#='C3' AND

grade < (SELECT AVG(grade)

FROM SC

Where C#='C3')

3.10 Insert, Update, and Delete



■ The Delete Statement

DELETE FROM tablename

[WHERE search_condition];

1. 删除单个元组

- **Example1:** 删除学号为S3的学生记录

Delete From S Where S#='S3'

- **EXAMPLE3.10.6:** Delete all agents in New York.

delete from agents where city='New York'

2、删除多个元组

- **Example2:** 删除所有学生的选课记录

Delete From SC

3.10 Insert, Update, and Delete



■ The Delete Statement

3、删除带子查询学生记录(在介绍了select后再讲)

- 例：把C3课程中小于该课程平均成绩的成绩记录从基本表SC中删去

Delete from SC

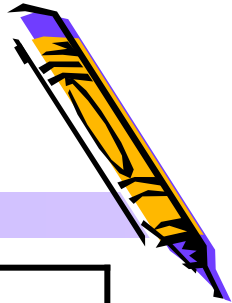
Where C#='C3' AND

grade < (SELECT AVG(grade)

FROM SC

Where C#='C3')

review



SQL功能		动词
SQL DDL		CREATE、DROP、ALTER
SQL DML	数据更新	INSERT、UPDATE、DELETE
	数据查询	SELECT

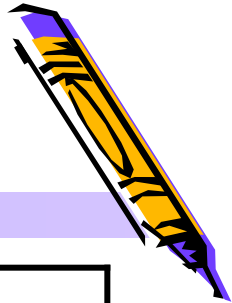
■ Database (definition)

- CREATE DATABASE ... ON ... FILEGROUP ... LOG ON
- ALTER DATABASE ... [ADD | MODIFY | REMOVE]
- DROP DATABASE

■ Table (definition)

- CREATE TABLE
- ALTER TABLE...[ADD | ALTER | DROP]
- DROP TABLE

review



SQL功能		动词
SQL DDL		CREATE、DROP、ALTER
SQL DML	数据更新	INSERT、UPDATE、DELETE
	数据查询	SELECT

■ Table (manipulate)

- INSERT INTO... VALUES...
- UPDATE SET...[WHERE]...
- DELETE FROM...[WHERE]...

- SELECT ...

3.3 Simple Select Statements



- **EXAMPLE 3.3.1** find the aid values and names of agents that are based in New York.
 - (AGENTS where city='New York')[aid,aname]
 - **SELECT** aid,aname **FROM** AGENTS **WHERE** city='New York'
- Start with the table following the word **FROM**.
- Performs the selection specified after the word **WHERE**.
- Extracts the projection defined by the field name(s) that follow the word **SELECT**.

3.3 Simple Select Statements



```
SELECT [ALL | DISTINCT | TOP N] { * | expr [[as]  
    c_alias]{,expr [[as] c_alias]...}}  
FROM tablename [[as] corr_name] {, tablename  
    [[as] corr_name]...}  
[WHERE search_condition];
```

- **Project** the resulting table on the attributes that appear in the **SELECT** list.
- Compute the **product** of the tables mentioned in the list that appears after the word **FROM**
- Apply the **selection** specified by the condition that follows **WHERE**

3.3 Simple Select Statements



- **EXAMPLE3.3.2** displays all the values in every row of the **CUSTOMERS** in order to check that the table load has taken place properly.

- **SELECT * FROM CUSTOMERS;**

- The symbol ***** is a shorthand symbol meaning retrieve all fields.

3.3 Simple Select Statements



- **EXAMPLE 3.3.3** retrieve all pid values of parts for which orders are placed.
 - **SELECT** pid **FROM** ORDERS;
 - **SELECT distinct** pid **FROM** ORDERS;
- The reserved word **distinct** actually guarantees that each row retrieved is unique .

3.3 Simple Select Statements



- Not obey Relational RULE 3
- But we can not Lost information
- Emphasize that all rows of a result are printed, that duplicates are not to be dropped, use the reserved word **ALL**.
 - **SELECT all pid FROM ORDERS;**
- **ALL** is the default option.

3.3 Simple Select Statements



- **EXAMPLE 3.3.4** retrieve all customer-agent name pairs(cname,aname), where the customers places an order through the agent.
 - $((\text{CUSTOMERS}[\text{cid}, \text{cname}] \bowtie \text{ORDERS}) \bowtie \text{AGENTS})$
[cname,aname]
 - $((\text{CUSTOMERS} \times \text{ORDERS} \times \text{AGENTS})$ Where
CUSTOMERS.cid=ORDERS.cid and
ORDERS.aid=AGENTS.aid) [cname, aname]

3.3 Simple Select Statements



■ **EXAMPLE 3.3.4** retrieve all customer-agent name pairs(cname,aname), where the customers places an order through the agent.

■ **SELECT** distinct CUSTOMERS.cname,
AGENTS.aname
FROM CUSTOMERS, ORDERS, AGENTS
WHERE CUSTOMERS.cid=ORDERS.cid and
ORDERS.aid=AGENTS.aid;

Database-students



- 学生 S(S#, SNAME, AGE, SEX)
- 课程 C(C#, CNAME, TEACH)
- 成绩 SC(S#, C#, GRADE)

s#	sname	age	sex
1	小张	18	男
2	小李	18	女
3	小王	19	男
4	小吴	19	女
5	小新	17	男
6	小赖	17	男
7	小毛	18	女

c#	cname	teach
c1	数据库原理与实用技术	zhxuan
c2	电子商务与电子政务	zhxuan
c3	C++程序设计	wangx

s#	c#	grade
1	c1	<NULL>
1	c3	<NULL>
2	c2	<NULL>
3	c1	<NULL>
3	c2	<NULL>
3	c3	<NULL>
4	c2	<NULL>
4	c3	<NULL>
5	c1	<NULL>
5	c3	<NULL>
6	c1	<NULL>

3.3 Simple Select Statements



- **EXAMPLE 1:** 查询全体学生的学号与姓名

```
select s#,sname  
from s
```

- **EXAMPLE 2:** 查询全体学生的详细记录

```
select *  
from s
```

- **EXAMPLE 3:** 查询全体学生的姓名及其出生年份

```
select sname,2010-age  
from s
```

3.3 Simple Select Statements



■ EXAMPLE 4:

```
select all sex,age  
from s
```

■ EXAMPLE 5:

```
select distinct sex,age  
from s
```

■ EXAMPLE 5:

```
select TOP 4*  
from s
```

3.3 Simple Select Statements



■ EXAMPLE 6: 查询全体男学生的名单

```
Select * from s  
where sex='男'
```

■ EXAMPLE 7: 查询所有年龄在18岁以下的学生姓名及其年龄

```
Select sname, age from s  
where age<18
```

等价于:

```
Select sname, age  
from s  
where not age>=18
```


3.3 Simple Select Statements



- **Example 3.3.5:** retrieve a “table” based on the orders table, with columns ordno, cid, aid, pid, and profit, where profit is calculated from qty and price of the product sold by subtracting 60% for wholesale cost, the discount for the customer, and the percent commission for the agent.

```
select ordno, x.cid, x.aid, x.pid, .40*(x.qty*p.price)-  
.01*(c.discnt+a.percent)*(x.qty*p.price)
```

```
from orders as x, customers as c, agents as a, products as p
```

```
where c.cid=x.cid and a.aid=x.aid and p.pid=x.pid;
```

3.3 Simple Select Statements



- **Example 3.3.6:** determine all pairs of customers based in the same city.

```
select c1.cid, c2.cid
```

```
from customers as c1, customers as c2
```

```
where c1.city=c2.city and c1.cid<c2.cid;
```

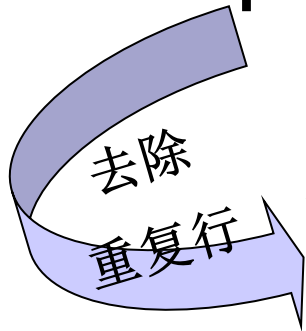
3.3 Simple Select Statements



■ Example 3.3.7

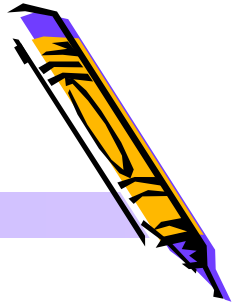
- find pid values of products that have been ordered by at least two customers.

```
select x1.pid  
from orders x1, orders x2  
where x1.pid=x2.pid and x1.cid<x2.cid
```



```
select distinct x1.pid  
from orders x1, orders x2  
where x1.pid=x2.pid and x1.cid<x2.cid
```

3.3 Simple Select Statements



■ Example 3.3.8

- Get cid values of customers who order a product for which an order is also placed by agent a06.
- ✓ First, consider the products for which an order is placed by agent a06

```
select x.pid from orders x where x.aid='a06';
```

- ✓

```
select distinct y.cid  
  from orders x, orders y  
 where y.pid=x.pid and x.aid='a06'
```

3.4 Subqueries



- A select statement form appearing within another select statement is known as a **subquery**, it can appear in the search_condition of a **WHERE** clause.
 - The **IN** predicate
 - The **quantified comparison** predicate
 - The **EXISTS** predicate
 - A weakness of SQL: too many **equivalent forms**

3.3 Simple Select Statements



■ The IN Predicate

■ 确定集合

- 谓词 **IN** 和 **NOT IN** 可以用来查找属性值属于(或不属于)指定集合的元组。
- 例：查询计算机系和外语系学生的姓名和性别。

Select sname, sex

from S

where dept **IN** ('计算机', '外语')

	s#	sname	age	sex	dept
1	1	小张	18	男	计算机
2	2	小李	18	女	工商管理
3	3	小王	19	男	计算机
4	4	小吴	19	女	计算机
5	5	小新	17	男	旅游
6	6	小赖	17	男	旅游
7	7	小毛	18	女	外语

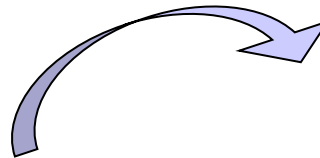
3.4 Subqueries



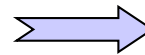
■ The IN Predicate

- 在嵌套查询中，子查询的结果往往是一个集合，所以谓词**IN** 是嵌套查询中最经常使用的谓词。
- 例1：查询与“小王”在同一个系学习的学生。

```
Select *  
from S  
where dept IN  
    ( Select dept  
      from S  
      where sname='小王' )
```



	s#	sname	age	sex	dept
1	1	小张	18	男	计算机
2	3	小王	19	男	计算机
3	4	小吴	19	女	计算机



Dept
计算机

- 如果不用**subquery**该怎么做？

3.4 Subqueries



■ The IN Predicate

■ 例2：查询选修了课程名为“数据库原理与实用技术”的学生信息

- 分析：学生信息涉及S表

```
Select s#, sname  
from S  
where s# in
```

```
( Select s#  
  from SC  
  where c# in  
    ( Select c#  
      from C  
      where cname='数据库原理与实用技术')  
)
```

deliver a set of rows to the
outer Select phrase without
receiving any input data,
these are Known as
uncorrelated Subqueries.

求解顺序



■ 如果不用subquery该怎么做？

3.4 Subqueries



- Example 3.4.1 get cid values of customers who place orders with agents in Duluth or Dallas.

Select cid

from orders

where aid IN

(select aid

from agents

where city='Duluth' or city='Dallas')

	cid
1	c001
2	c004
3	c001
4	c006
5	c001
6	c002

	aid
1	a05
2	a06

3.4 Subqueries



- Example 3.4.2 retrieve all information concerning agents based in Duluth or Dallas.

```
select *  
  from agents  
 where city IN ('Duluth' , 'Dallas')
```

	aid	aname	city	percent
1	a05	Otasi	Duluth	5
2	a06	Smith	Dallas	5

3.4 Subqueries



- Example 3.4.3 determine the names and discounts of all customers who place orders through agents in Duluth or Dallas.

Select cname, discnt from customers

where cid in

(select cid from orders

where aid IN

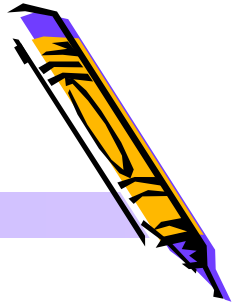
(select aid from agents

where city='Duluth' or city='Dallas')

)

	cname	discnt
1	TipTop	10.00
2	Basics	12.00
3	ACME	8.00
4	ACME	.00

3.4 Subqueries



■ The IN Predicate

- A Subquery using data from an outer Select is known as a *correlated Subquery*.
- Example 3.4.4 find the names of customers who order product p05

```
Select cname
from customers
where 'p05' in
( Select pid
  from orders
  where cid=customers.cid)
```

求解顺序

	cname
1	Allied
2	TipTop

3.4 Subqueries



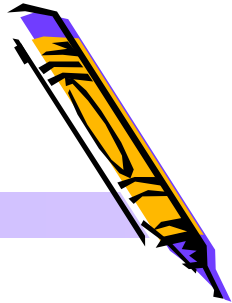
■ The IN Predicate

- A Subquery using data from an outer Select is known as a *correlated Subquery*.
- Example 3.4.5 Get the name of customers who order product p07 from agent a03

```
Select cname
from customers
where orders.aid='a03' and 'p07' in
( Select pid
  from orders
  where cid=customers.cid)
```

作用域问题

3.4 Subqueries



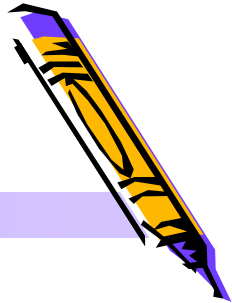
■ The IN Predicate

■ Example 3.4.5 uncorrelated Subqueries

```
Select cname
from customers
where cid in
( Select cid
  from orders
  where pid='p07' and aid='a03')
```

	cname
1	ACME

3.4 Subqueries



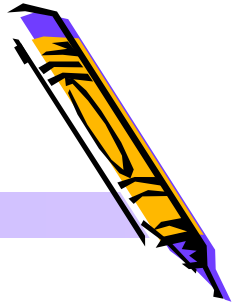
■ The IN Predicate

- Example 3.4.6 retrieve ordno values for all orders placed by customers in Duluth through agents in New York.

```
Select ordno
from orders
where (cid,aid) in
      (Select cid,aid
       from customers c, agents a
       where c.city='Duluth' and a.city='New York')
```

- DB2、ORACLE

3.4 Subqueries



■ The IN Predicate

- Example 3.4.6 retrieve ordno values for all orders placed by customers in Duluth through agents in New York (SQL Server 2000).

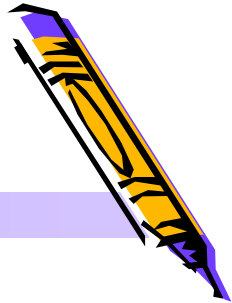
```
Select ordno  
from orders  
where cid in
```

	ordno
1	1011
2	1012
3	1023

```
(Select cid from customers where city='Duluth')  
and aid in
```

```
(select aid from agents where city='New York')
```


3.4 Subqueries



■ The Quantified Comparison Predicate

- 父查询与子查询之间用比较运算符进行连接。当用户能确切知道内层查询返回的是单值时，可以用>，<，>=，<=，!=或<>等比较运算符。
- 例3：与“小王”在同一个系学习的学生(由于一个学生只能在一个系学习，也就是说内查询的结果是一个值，因此可以用 = 代替IN)，其SQL语句如下：

```
Select s#, sname, dept
from S
where dept =
    ( Select dept
      from S
      where sname='小王' )
```

3.4 Subqueries



■ The Quantified Comparison Predicate

- 子查询返回单值时可以用比较运算符，而使用**ANY** | **SOME**或**ALL**谓词时则必须同时使用比较运算符。其语义为：

>ANY SOME	大于子查询结果中的某个值
>ALL	大于子查询结果中的所有值
< ANY SOME	小于子查询结果中的某个值
< ALL	小于子查询结果中的所有值
>=ANY SOME	大于等于子查询结果中的某个值
>=ALL	大于等于子查询结果中的所有值
...	...

3.4 Subqueries



■ The Quantified Comparison Predicate

- 例4：查询其他系中比计算机系某一学生年龄小的学生学号、姓名、年龄和所在系。

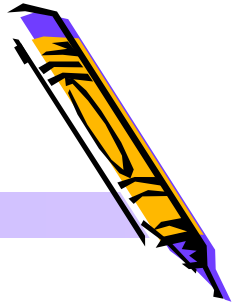
```
Select s#, sname, age, dept
from S
where age < SOME
      ( select age from S
        where dept = '计算机' )
and dept <> '计算机'
```

age
18
19
19

查询不是计算机系，
且年龄小于**18**、或
19岁的学生

	s#	sname	age	dept
1	2	小李	18	工商管理
2	5	小新	17	旅游
3	6	小赖	17	旅游
4	7	小毛	18	外语

3.4 Subqueries



■ The Quantified Comparison Predicate

- 例5：查询其他系中比计算机系所有学生年龄小的学生学号、姓名、年龄和所在系。

```
Select sno, sname, age, dept
from student
where age < ALL
      ( select age from student
        where dept = '计算机' )
and dept <> '计算机'
```

age
18
19
19

	s#	sname	age	dept
1	5	小新	17	旅游
2	6	小赖	17	旅游

查询不是计算机系，且年龄小于18、岁的学生

3.4 Subqueries



■ The Quantified Comparison Predicate

- Example 3.4.7 find aid values of agents with a minimum percent commission.

```
select aid
from agents
where [percent] <= all
      (select [percent] from agents)
```

	aid
1	a05
2	a06

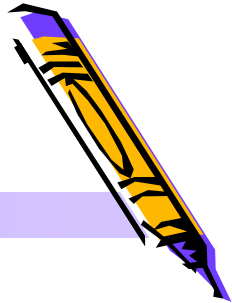
3.4 Subqueries



■ The Quantified Comparison Predicate

- **Example 3.4.8** find all customers who have the same discount as that of any of the customers in Dallas or Boston.
- **Example 3.4.9** get cid values of customers with discount smaller than those of any customers who live in Duluth.

3.4 Subqueries



■ The EXISTS Predicate

- 带有**EXISTS**谓词的子查询不返回任何数据，只产生逻辑真值“**true**”或逻辑假值“**false**”。

- 例6：查询所有选修了c1课程的学生学号和姓名

Select s#, sname

from S

where EXISTS

(select * . . .

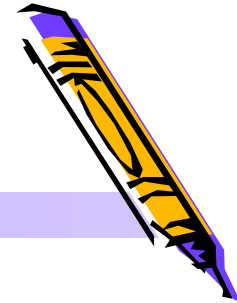
from SC

where SC.s#=S.s# and c#='c1')

存在量词**Exists**并不关心内层查询的值是多少，因此，内层循环通常不指定列名

- 使用存在量词**EXISTS**后，若内层查询结果非空，则外层的**WHERE**子句返回真值，否则返回假值

3.4 Subqueries



■ The EXISTS Predicate

- 与**EXISTS**谓词对应的是**NOT EXISTS**。
- 例7：查询所有没有选修**c1**课程的学生学号和姓名

Select s#, sname

from S

where NOT EXISTS

(select *

from SC

where SC.s#=S.s# and c#='c1')

	s#	sname
1	2	小李
2	7	小毛
3	4	小吴

3.4 Subqueries



■ The EXISTS Predicate

- **Example 3.4.10** retrieve all customer names where the customer places an order through agent a05.

Select cname from customers

where cid in

(Select cid from orders where aid='a05')

- **Example 3.4.12** retrieve all customer names where the customer does not place an order through agent a05.

3.5 UNION Operators and FOR ALL Conditions



■ Division: SQL “FOR ALL...” Conditions

- **Example 3.5.2** Get the cid values of customers who place orders with all agents based in New York.

ORDERS [*cid*, *aid*] $\div$ (*AGENTS* where *city* = 'New York')[*aid*]

```
cond1:  a.city = 'New York' and  
        not exists (select * from orders x  
                    where x.cid = c.cid and x.aid = a.aid)
```

存在住在New York的代理商a.aid，他没有给顾客c.cid供货

3.5 UNION Operators and FOR ALL Conditions



■ Division: SQL “FOR ALL...” Conditions

- **Example 3.5.2** Get the cid values of customers who place orders with all agents based in New York.

```
cond2:  not exists (select * from agents a where cond1)
```

或者写成完整的形式:

```
cond2:  not exists (select * from agents a where a.city = 'New York'
and not exists (select * from orders x
where x.cid = c.cid and x.aid = a.aid))
```

不存在这样的一个住在New York的代理商a.aid, 他没有为c.cid订货。就意味着住在New York的所有代理商都为c.cid订了货。

3.5 UNION Operators and FOR ALL Conditions



■ Division: SQL “FOR ALL...” Conditions

- **Example 3.5.2** Get the cid values of customers who place orders with all agents based in New York.
- The answer to this request is given by::

```
select c.cid from customers c where
    not exists ( select * from agents a
                  where a.city='New York' and
                        not exists ( select * from orders x
                                    where x.cid=c.cid and x.aid=a.aid))
```

	cid
1	c001

3.5 UNION Operators and FOR ALL Conditions



■ Division: SQL “FOR ALL...” Conditions

- **Example 3.5.3** Get the aid values of agents in New York or Duluth who place orders for all products costing more than a dollar.

```
select aid from agents a
where (a.city = 'New York' or a.city = 'Duluth')
and not exists (select p.pid from products p
                where p.price > 1.00 and not exists (select * from orders x
                where x.pid = p.pid and x.aid = a.aid));
```

选择满足条件的代理商的
aid

aid
a05

3.5 UNION Operators and FOR ALL Conditions



■ More example:

- **Example 3.5.4** find aid values of agents who place orders for product p01 as well as for all products costing more than a dollar.
- **Example 3.5.5** find cid values for customers with the following property: if customer c006 orders a particular product, so does the customer under consideration.
- **Example 3.5.6** find pid values of products supplied to all customers in Duluth.

3.5 UNION Operators and FOR ALL Conditions



■ The UNION Operator

■ Subquery UNION [ALL] Subquery

- Example 3.5.1 create a list of cities where either a customer or an agent, or both, is based.

Select city from customers

union [ALL] Select city from agents;

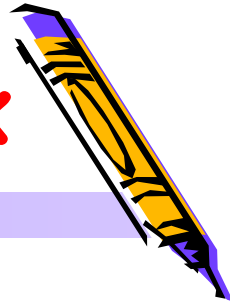
- Look at the difference:

(Select city from customers union Select city from agents)

union all select city from products;

Select city from customers union (Select city from agents
union all select city from products);

3.6 Some Advanced SQL Syntax



□ The **INTERSECT** and **EXCEPT** Operators

Subquery **INTERSECT** [ALL] | **EXCEPT** [ALL] **Subquery**

□ Example 3.6.1 requesting cid values of customers who order both products p01 and p07

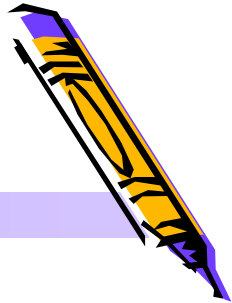
Select cid from orders where pid='p01'

INTERSECT select cid from orders where pid='p07'

□ Example 3.6.2 retrieve all customer names where the customer does not place an order through agent a05.

EXCEPT...NOT EXISTS

3.6 Some Advanced SQL Syntax



■ INNER、OUTER JOIN

■ **FROM**子句の格式:

FROM {<joined_table>}

<joined_table>::=

<table_source> <join_type> <table_source> **ON** <search_condition>

<join_type>::=

[INNER|{{LEFT | RIGHT | FULL} [OUTER]}] JOIN

3.6 Some Advanced SQL Syntax

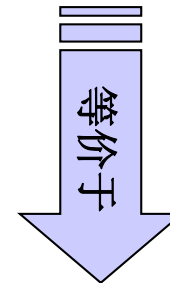


■ INNER JOIN

- **INNER** 联接基于列值之间的相等性，只有两个表之间联接列数值相等的行才在结果集中返回。如果不指定联接的操作类型，则默认为 **INNER**。

- 例5：查询学生的学号、姓名和课程成绩

```
Select S.s#, sname, c#, grade  
from S, SC  
where S.s#=sc.s#
```



```
Select S.s#, sname, c#, grade  
from S [INNER] JOIN SC ON S.s#=SC.s#
```

3.6 Some Advanced SQL Syntax



■ INNER JOIN

- Example 3.6.4 retrieve all customers name who purchased at least one product costing less than \$1.00.

select distinct cname

from (orders o join products p on o.pid=p.pid)

join customers c on o.cid=c.cid

where p.price<1

	cname
1	ACME
2	TipTop

3.6 Some Advanced SQL Syntax



LEFT OUTER JOIN

■ **OUTER**返回指定外部表中的所有行（用**LEFT OUTER**、**RIGHT OUTER**、**FULL OUTER**指定）。如果外部表中没有匹配的行，则对应列显示**NULL**。

■ 例6：查询所有学生的选课信息（包括没有选课的学生）

Select S.s#, sname, c#, grade

from S LEFT OUTER JOIN SC ON S.s#=SC.s#

from S RIGHT OUTER JOIN SC ON S.s#=SC.s#

	s#	sname	c#	grade
1	6	小赖	c1	97.00
2	2	小李	c2	89.00
3	7	小毛	NULL	NULL
4	3	小王	c1	77.00
5	3	小王	c2	97.00
6	3	小王	c3	90.00
7	4	小吴	c2	78.00
8	4	小吴	c3	98.00
9	5	小新	c1	68.00
10	5	小新	c3	54.00
11	1	小张	c1	90.00
12	1	小张	c3	99.00

	s#	sname	c#	grade
1	1	小张	c1	90.00
2	1	小张	c3	99.00
3	2	小李	c2	89.00
4	3	小王	c1	77.00
5	3	小王	c2	97.00
6	3	小王	c3	90.00
7	4	小吴	c2	78.00
8	4	小吴	c3	98.00
9	5	小新	c1	68.00
10	5	小新	c3	54.00
11	6	小赖	c1	97.00

3.6 Some Advanced SQL Syntax



■ FULL OUTER JOIN

■ **FULL OUTER JOIN**全外联接，返回两个表中所有匹配的行，以及对方表中没有匹配的行。

■ **例6**：查询所有学生的选课信息（包括没有选课的学生也包括选课了未登记进**S**表的学生）

Select S.s#, sname, c#, grade

from S **FULL OUTER JOIN** SC **ON** S.s#=SC.s#

	s#	sname	c#	grade
1	1	小张	c1	90.00
2	1	小张	c3	99.00
3	2	小李	c2	89.00
4	3	小王	c1	77.00
5	3	小王	c2	97.00
6	3	小王	c3	90.00
7	4	小吴	c2	78.00
8	4	小吴	c3	98.00
9	5	小新	c1	68.00
10	5	小新	c3	54.00
11	6	小赖	c1	97.00
12	7	小毛	NULL	NULL

3.7 Set Functions in SQL



- Set Function (in DB2 Universal Database, uses the term *column functions*)
 - SQL provides five built-in functions that operate on sets of column values in table: **COUNT**, **MAX**, **MIN**, **SUM**, and **AVG**.

COUNT ([DISTINCT ALL] *)	统计元组个数
COUNT ([DISTINCT ALL]<列名>)	统计一列中值的个数
SUM ([DISTINCT ALL]<列名>)	计算一列值的总和（此列必须是数值型）
AVG ([DISTINCT ALL]<列名>)	计算一列值的平均值（此列必须是数值型）
MAX ([DISTINCT ALL]<列名>)	求一列值中的最大值
MIN ([DISTINCT ALL]<列名>)	求一列值中的最小值

3.7 Set Functions in SQL



- 例1: 查询学生总人数

Select **count(*)** from S

- 例2: 查询选修了课程的学生人数

Select **count(distinct s#)** from SC

- 例3: 计算C3号课程的学生平均成绩

Select **AVG(grade)** from SC where c#='C3'

3.7 Set Functions in SQL



- Example 3.7.1 To determine the total dollar amount of all orders

Select **sum(dollars)** as total from orders

- Example 3.7.2 To determine the total quantity of product p03 that has been ordered.

Select **sum(qty)** from orders where pid='p03'

- Example 3.7.4 get the number of cities where customers are based?

Select **count(city)** from customers

Select **count(distinct city)** from customers

3.8 Group of Rows in SQL



■ GROUP BY 子句一对查询结果进行分组

- **GROUP BY**子句将查询结果表按一列或多列值分组，值相等的为一组。

- 格式：

[**GROUP BY** [**ALL**] group_by_expression[,...n]]

- 例2：查询各门课程的选课人数

```
Select C#,count(distinct s#) from SC
      group by c#
```

- Example3.7.1 To determine the total order dollar amount of all products

```
Select pid,sum(dollars) as total from orders
      group by pid
```

3.8 Group of Rows in SQL



- 例2：查询各门课程的课程号，课程名和相应的选课人数

```
select sc.c#,cname,count(distinct s#)  
from sc join c on sc.c#=c.c#  
group by sc.c#,cname
```



3.8 Group of Rows in SQL

■ GROUP BY 子句—对查询结果进行分组

■ **ALL**表示包括所有的组和结果，即使是不符合**WHERE**子句指定条件的分组也将列出，但并不对这些分组进行统计。

■ 例2：求C1和C3课程相应的选课人数

```
Select c#, COUNT(s#)
```

```
from SC
```

```
where c# in ('c1', 'c3')
```

```
GROUP BY all c#
```

	c#	(无列名)
1	c1	4
2	c2	0
3	c3	4

3.8 Group of Rows in SQL



- Note that the GROUP BY clause is written following the WHERE clause
 - ◆ First the relational products of all tables in the FROM clause are formed.
 - ◆ From this, row not satisfying the WHERE clause are eliminated.
 - ◆ The remaining rows are grouped in accordance with the GROUP BY clause.
 - ◆ Finally, expressions in the select list are evaluated.

3.8 Group of Rows in SQL



- [例] Print out all product and agent IDs and the total quantity ordered of the product by the agent when this quantity exceeds 1000.

◆ Select pid, aid, sum(qty)
from orders
where **sum(qty)>1000**
group by pid, aid;

--** INVALID SQL SYNTAX: A set
function cannot occur in the WHERE clause.

3.8 Group of Rows in SQL



- **HAVING**—To create a conditional depend on knowledge of this grouping, SQL Select statements are provided with a new restriction clause, known as the HAVING clause.

```
select pid, aid, sum(qty) as Total  
from orders  
group by pid, aid  
having sum(qty)>1000
```

3.8 Group of Rows in SQL



■ **HAVING**子句—在**GROUP BY**子句中帶条件的查询

■ **HAVING**与**WHERE**的区别

- **WHERE**子句应用于表中的个别行，将符合**WHERE**条件的行选出后再进行分组
- **HAVING**子句只能应用于**GROUP BY**子句之后，并且**HAVING**子句中的查询条件只能包含在**GROUP BY**子句中的列或集合函数

3.8 Group of Rows in SQL



■ **HAVING**子句—在**GROUP BY**子句中帶条件的查询

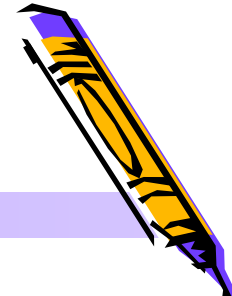
■ **HAVING**与**WHERE**的区别

- 例2：求选课人数超过3人的课程的课程号及相应的选课人数

```
Select c#, COUNT(s#)
from SC
GROUP BY c#
HAVING count(s#)>3
```

	s#	c#	grade
1	1	c1	90.00
2	1	c3	99.00
3	2	c2	89.00
4	3	c1	77.00
5	3	c2	97.00
6	3	c3	90.00
7	4	c2	78.00
8	4	c3	98.00
9	5	c1	68.00
10	5	c3	54.00
11	6	c1	97.00
..			

3.8 Group of Rows in SQL

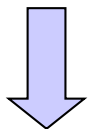


■ **HAVING**子句—在**GROUP BY**子句中帶条件的查询

■ **HAVING**与**WHERE**的区别

- 例3：求每门课程分数不低于80分的学生人数

```
Select c#, COUNT(s#)
from SC
where grade >= 80
GROUP BY c#
```



```
Select c#, COUNT(s#)
from SC
GROUP BY c#
HAVING grade >= 80
```

	s#	c#	grade
1	1	c1	97.00
9	5	c3	63.00
10	5	c3	54.00
11	6	c1	97.00
..			

grade在**HAVING**子句中无效，因为其既不包含在**GROUP BY**子句中，也不是聚合函数

3.8 Group of Rows in SQL



■ COMPUTE子句

- **COMPUTE**子句可以用于聚合函数（SUM、AVG、MIN、MAX和COUNT）生成分段的汇总值，汇总结果作为查询结果中的附加行出现。

- 例1：查询每门课程的选修情况和所有课程的总平均分

```
Select c#,s#,grade  
from sc  
compute avg(grade)
```

插入所有课程
的总平均分

- 例2：

```
Select *  
from orders  
compute avg(dollars)
```

3.9 A Complete Description of SQL Select



■ ORDER BY子句—对查询结果进行排序

- 用户可以用**ORDER BY**子句对查询结果按照一个或多个属性列的升序（**ASC**）或降序（**DESC**）排列，缺省值为升序

- 格式：**[ORDER BY {order_by_expression[ASC|DESC]}
[,...n]]**

- 例1：查询选修了C3课程的学生们的学号及其成绩，查询结果按分数的降序排列。

```
Select s#,grade from SC  
where c#='C3'  
order by grade DESC
```

- 例2：查询选修了C3课程的学生们的学号及其成绩，查询结果按分数的升序排列。

```
Select s#,grade from SC  
where c#='C3'  
order by grade
```

3.9 A Complete Description of SQL Select



■ WHERE子句一带条件的查询

■ 确定范围

- 谓词**BETWEEN ...AND...**和**NOT BETWEEN ...AND...**可以用来查找属性值在（或不在）指定范围内的元组。
- 例：查询年龄在18~19（包括18岁和19岁）之间的学生的姓名、系别和年龄。

```
Select sname, dept, age  
from S  
where age between 18 and 19
```

	s#	sname	age	sex	dept
1	1	小张	18	男	计算机
2	2	小李	18	女	工商管理
3	3	小王	19	男	计算机
4	4	小吴	19	女	计算机
5	5	小新	17	男	旅游
6	6	小赖	17	男	旅游
7	7	小毛	18	女	外语

3.9 A Complete Description of SQL Select



■ WHERE子句一带条件的查询

■ 字符匹配

- 谓词**LIKE**用于判断属性值是否与指定字符串格式匹配，可用于字符匹配的数据类型有**char**、**varchar**、**text**、**ntext**、**datetime**、**smalldatetime**等。
- 通配符

通配符	含义
%	任意字符
_	任意一个字符

3.9 A Complete Description of SQL Select



■ WHERE子句一带条件的查询

■ 字符匹配

- 例1: 查询学号为2的学生的详细情况。

```
Select * from S  
where s# LIKE '2'
```

等价于:

```
Select * from S  
where s# = '2'
```

- 例2: 查询所有叫某某吴的学生的姓名、学号和性别

```
Select sname, s#, sex from S  
where sname LIKE '%吴'
```

- 例3: 查询姓“欧阳”且全名为三个汉字的学生的姓名

```
Select sname from S  
where sname LIKE '欧阳_'
```

SQL Language – Select



■ INTO子句

- **SELECT**语句附加**INTO**子句后，不将结果直接输出，而是生成一个新表，并将数据输出到新表中
- 格式：**INTO new_table**
- 其中创建的**new_table**可以是一个永久性数据表，也可以是一个临时数据表
- 例如：查询学生的学号、姓名和所在系

```
Select s#, sname, dept  
Into student_into  
from S
```

```
Select s#, sname, dept  
Into tempdb.#student_into  
from S
```

3.7 Set Functions in SQL



■ Handling Null Values

■ 涉及空值（NULL）的查询

- 例1：查询缺少成绩的学生的学号和相应的课程号。

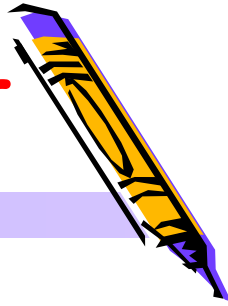
```
Select s#,c# from SC  
where grade IS NULL
```

注意：此处的'IS'不能用等号代替

- 例2：查询所有有成绩的学生学号和课程号

```
Select s#,c# from SC  
where grade IS NOT NULL
```


3.9 A Complete Description of SQL Select



[3.9.1] List all customers, agents, and the dollars sales for pairs of customers and agents, and order the result from largest to smallest sales totals. Retain only those pairs for which the dollar amount is at least equal to 900.00

```
select c.cname, c.cid, a.aname, a.aid, sum(o.dollars) as casales
  from customers c, orders o, agents a
 where c.cid = o.cid and o.aid = a.aid
group by c.cname, c.cid, a.aname, a.aid
having sum(o.dollars) >= 900.00
order by casales desc;
```

lab



■ 实验: lab5